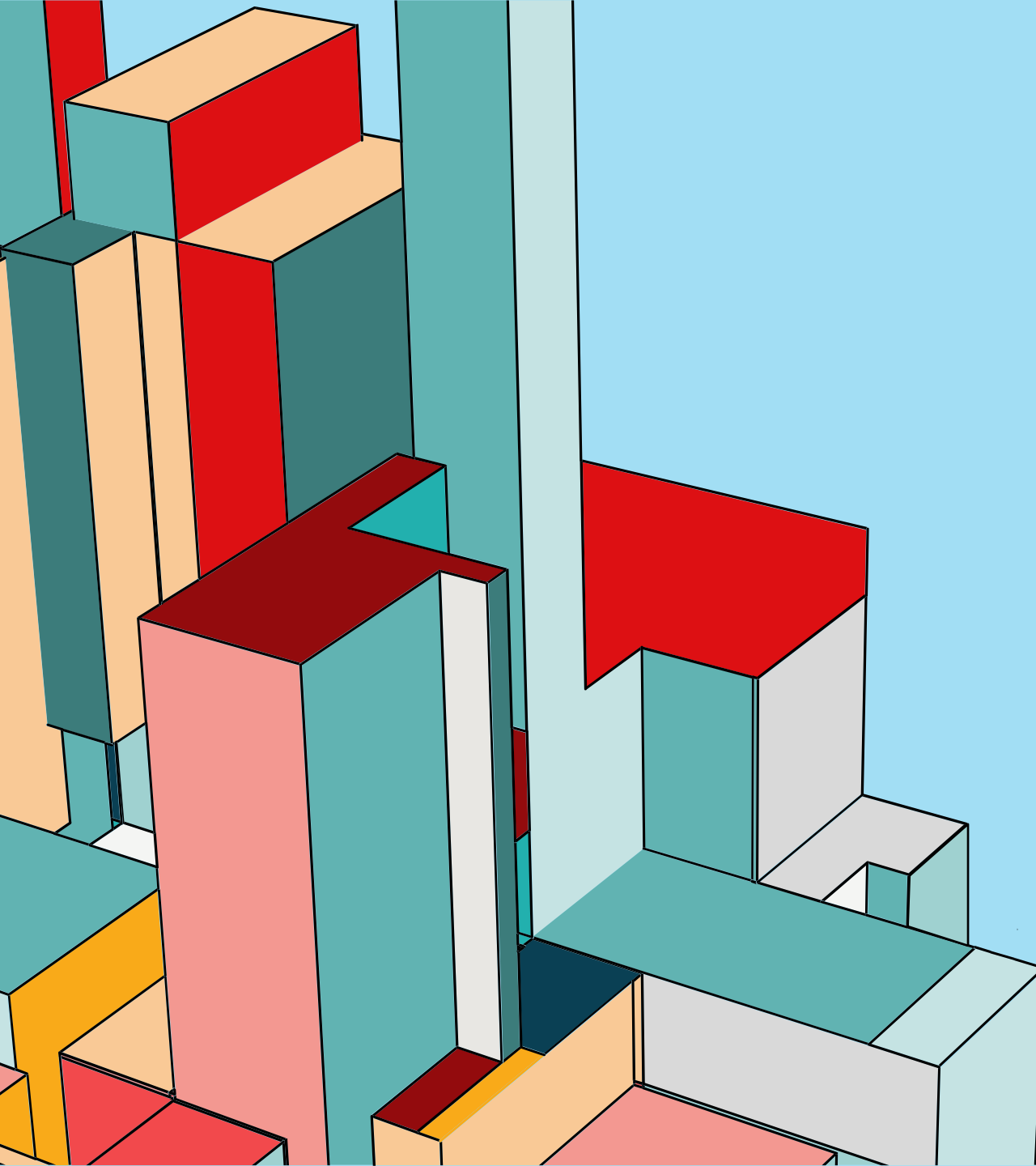
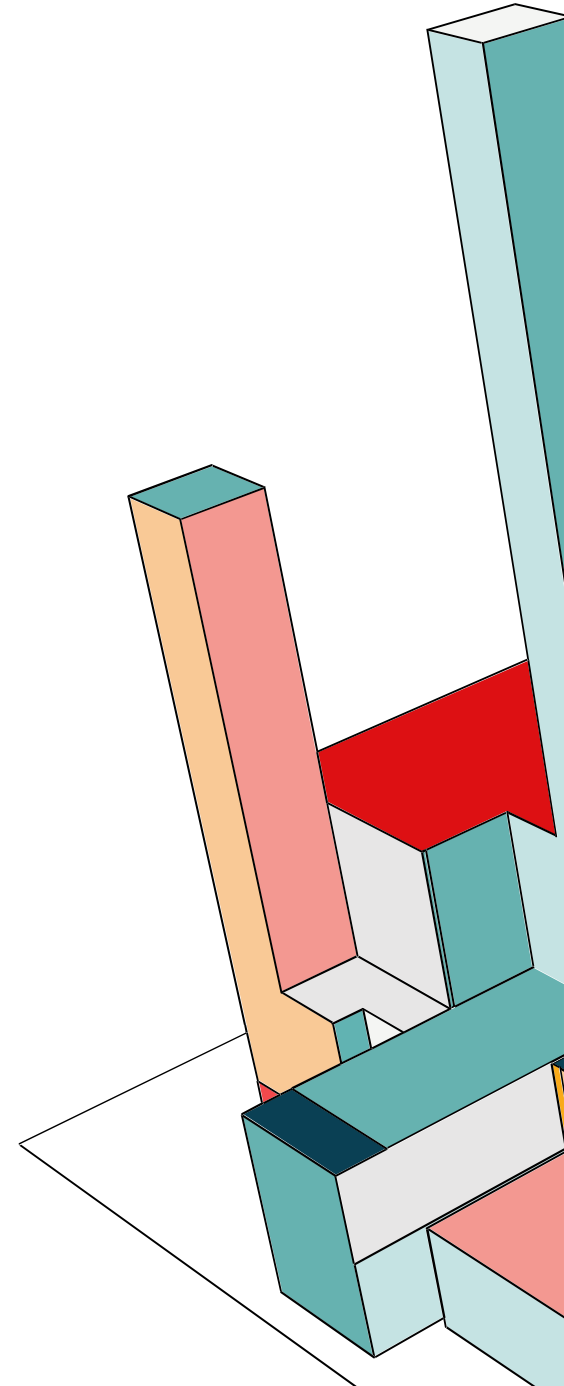




ARSITEKTUR APLIKASI WEB MODERN DAN MVC

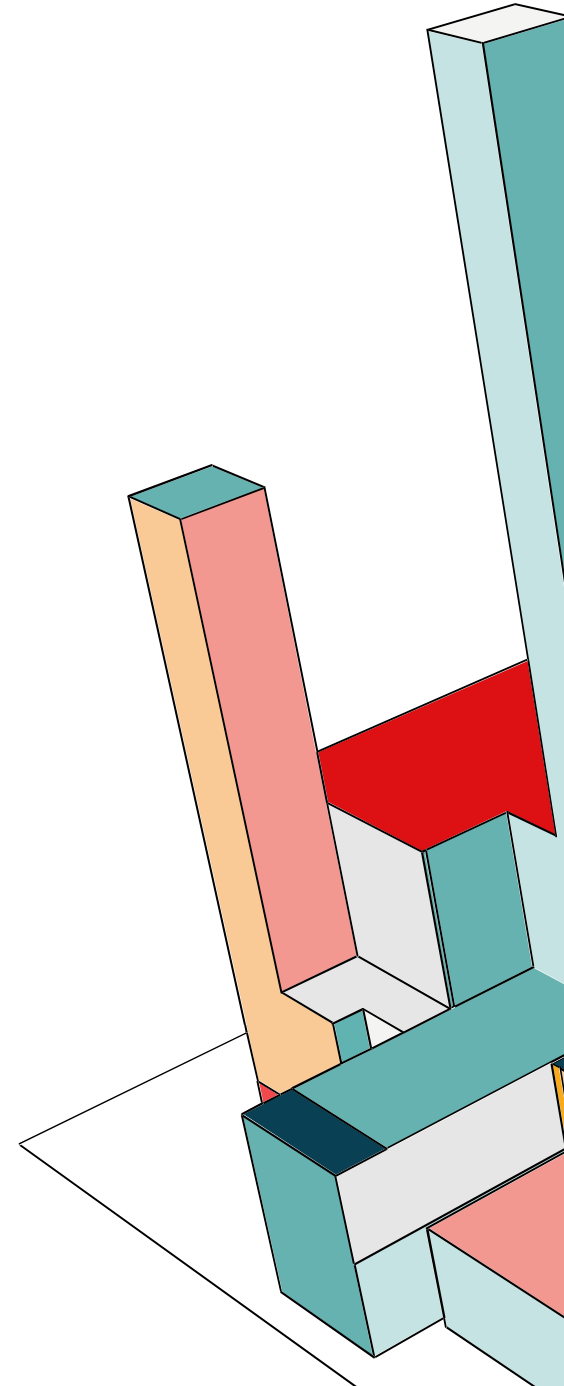
MINGGU 2

- **Mahasiswa mampu menyiapkan lingkungan pengembangan dan merencanakan proyek aplikasi web**
 - Praktikum di lab
 - Diskusi kelompok
 - Brainstorming proyek
 - Media: Komputer, IDE, Git, Project management tools
 - Sumber: Dokumentasi framework, Git documentation
- ***Memahami arsitektur aplikasi web modern dan MVC***
 - *Pengantar Arsitektur Aplikasi Web Modern*
 - *Model-View-Controller (MVC)*
 - *Implementasi MVC dalam Aplikasi Web Modern*
 - *Contoh Sederhana*



AGENDA

- Model Pengembangan Perangkat Lunak: Waterfall dan Agile
- Pengantar Arsitektur Aplikasi Web Modern
- Model-View-Controller (MVC)
- Implementasi MVC dalam Aplikasi Web Modern
- Alur Kerja MVC dalam Aplikasi Web
- Best Practices

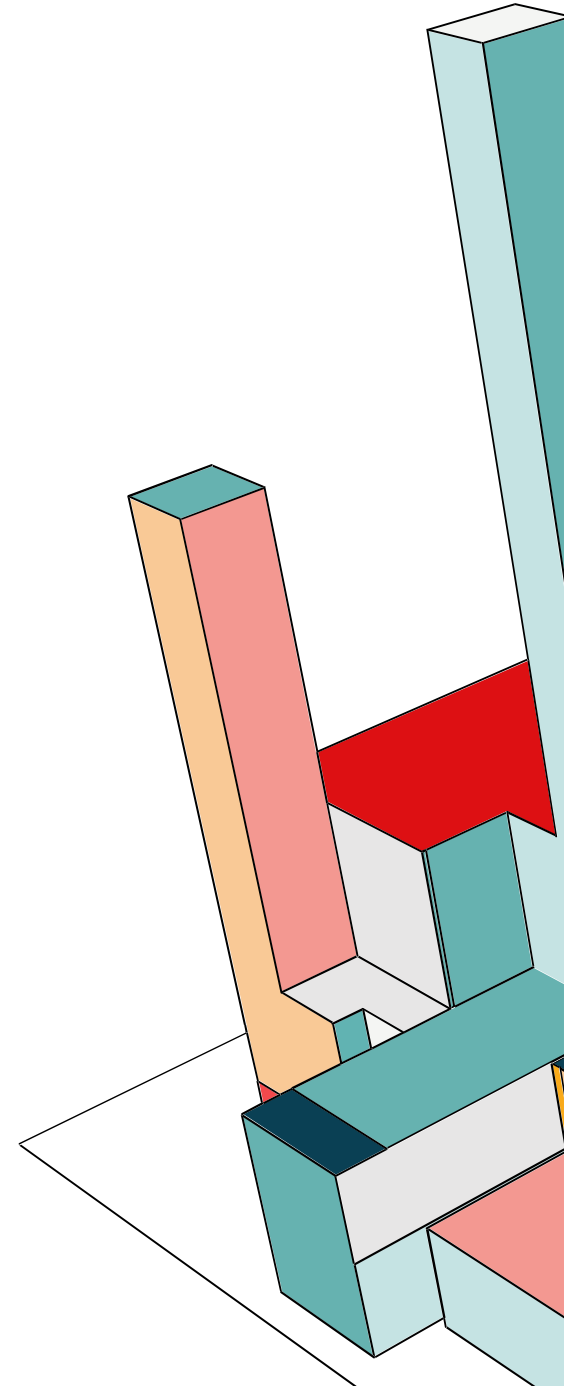


MODEL PENGEMBANGAN PERANGKAT LUNAK

Bagian 1 dari 5

Sebelum membahas arsitektur aplikasi, kita perlu memahami lebih dulu bagaimana sebuah perangkat lunak direncanakan dan dibangun dari waktu ke waktu.

- Waterfall — pendekatan prediktif
- Agile — pendekatan adaptif



MENGAPA PERLU MODEL PENGEMBANGAN?

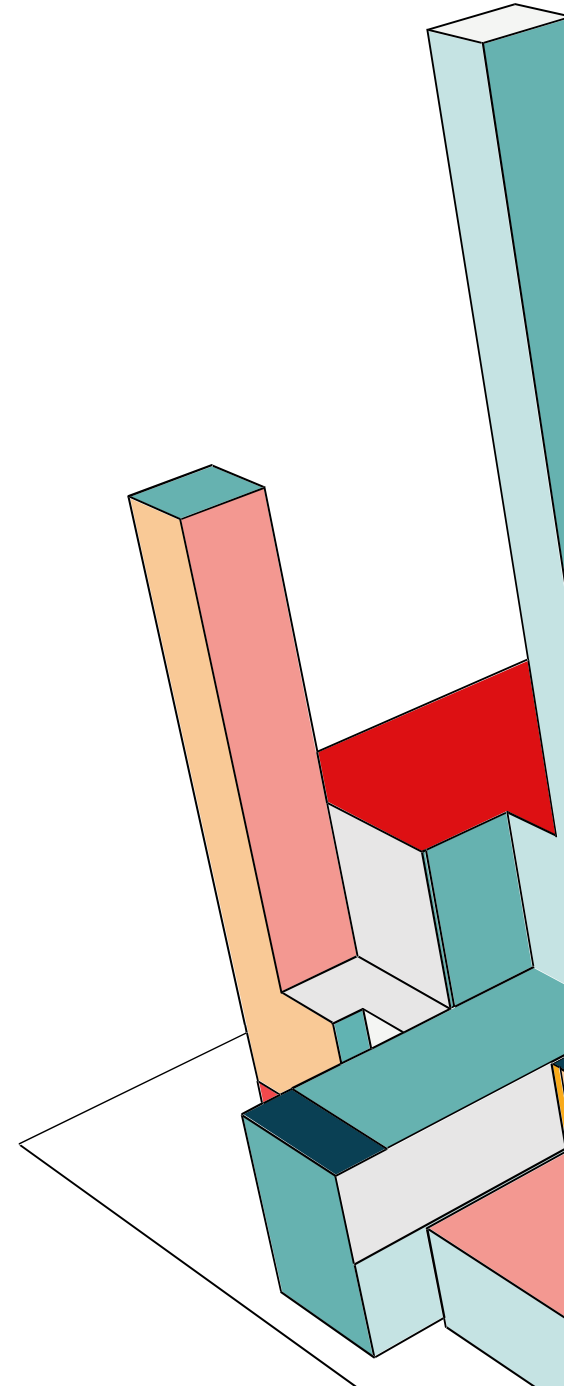
Pengembangan perangkat lunak melibatkan banyak orang, banyak berkas, dan kebutuhan yang terus berubah.

Tanpa model yang disepakati:

- Fitur dikerjakan tanpa urutan, saling menunggu dan saling menimpa
- Tidak ada kesepakatan kapan sebuah pekerjaan dianggap selesai
- Perubahan kebutuhan masuk tanpa kendali dan merusak jadwal

Model pengembangan (SDLC) menetapkan:

- Tahapan apa saja yang harus dilalui dan dalam urutan apa
- Siapa mengerjakan apa pada setiap tahap
- Kapan hasil diserahkan dan diuji



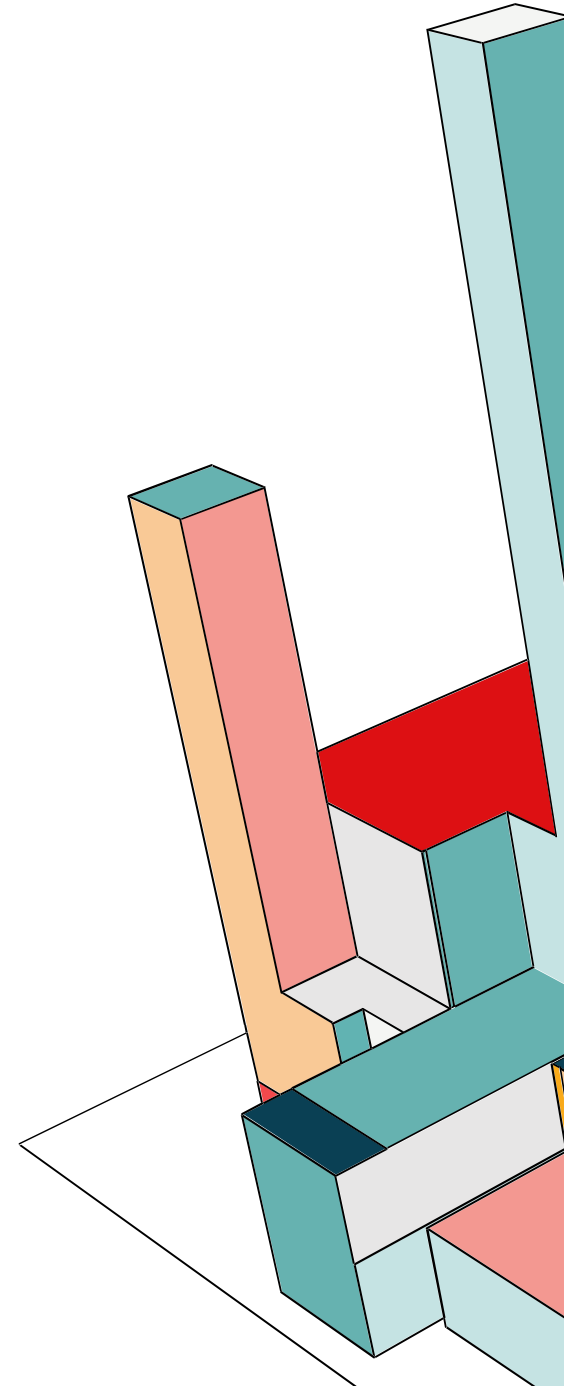
MODEL WATERFALL

Pendekatan linear: setiap tahap harus selesai sepenuhnya sebelum tahap berikutnya dimulai, seperti air terjun yang mengalir satu arah.

Tahapan Waterfall

1. Analisis Kebutuhan — seluruh kebutuhan dikumpulkan dan didokumentasikan
2. Perancangan — arsitektur, basis data, dan antarmuka dirancang
3. Implementasi — penulisan kode program
4. Pengujian — verifikasi terhadap dokumen kebutuhan
5. Deployment — perangkat lunak diserahkan ke pengguna
6. Pemeliharaan — perbaikan dan penyesuaian setelah digunakan

Ciri utama: kebutuhan dibekukan sejak awal, dokumentasi lengkap di muka, dan produk utuh baru terlihat pada akhir proyek.



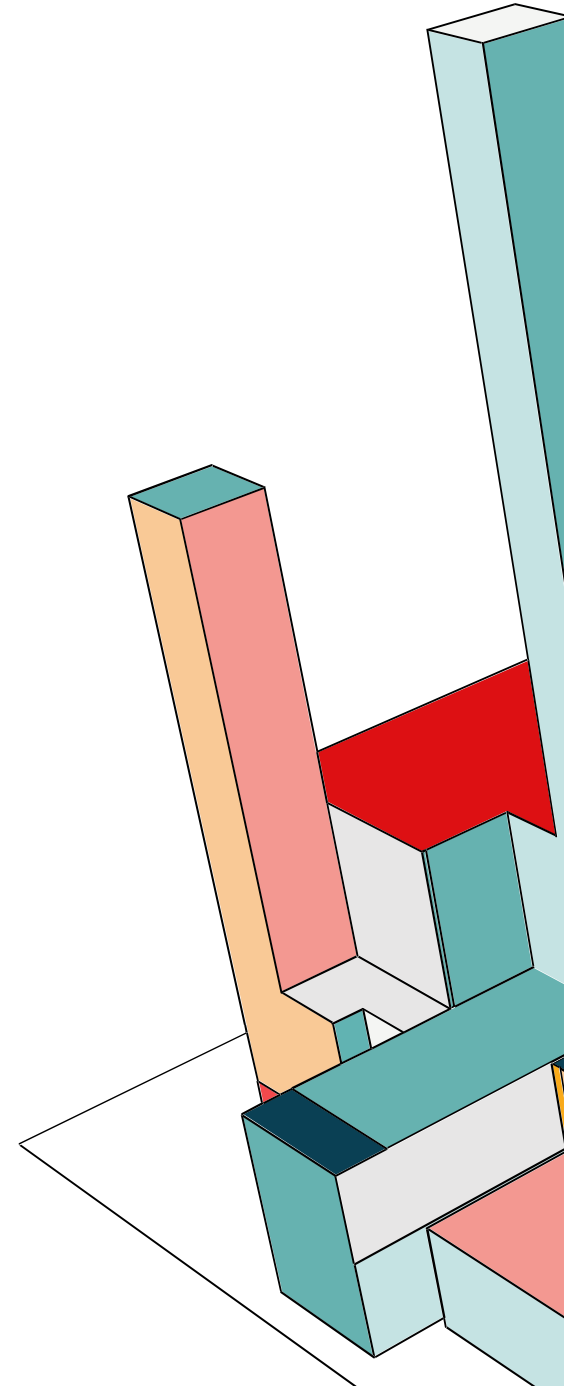
WATERFALL: KELEBIHAN DAN KEKURANGAN

Kelebihan

- Struktur jelas dan mudah dikelola karena tahapannya pasti
- Dokumentasi lengkap, memudahkan serah terima dan audit
- Perkiraan biaya dan waktu lebih mudah disusun di awal
- Cocok bila kebutuhan stabil dan terikat kontrak atau regulasi

Kekurangan

- Biaya perubahan sangat mahal setelah tahap berjalan
- Pengguna baru melihat produk di akhir, sehingga salah paham baru ketahuan terlambat
- Risiko teknis tersembunyi sampai tahap implementasi
- Tidak cocok untuk produk baru yang kebutuhannya belum pasti



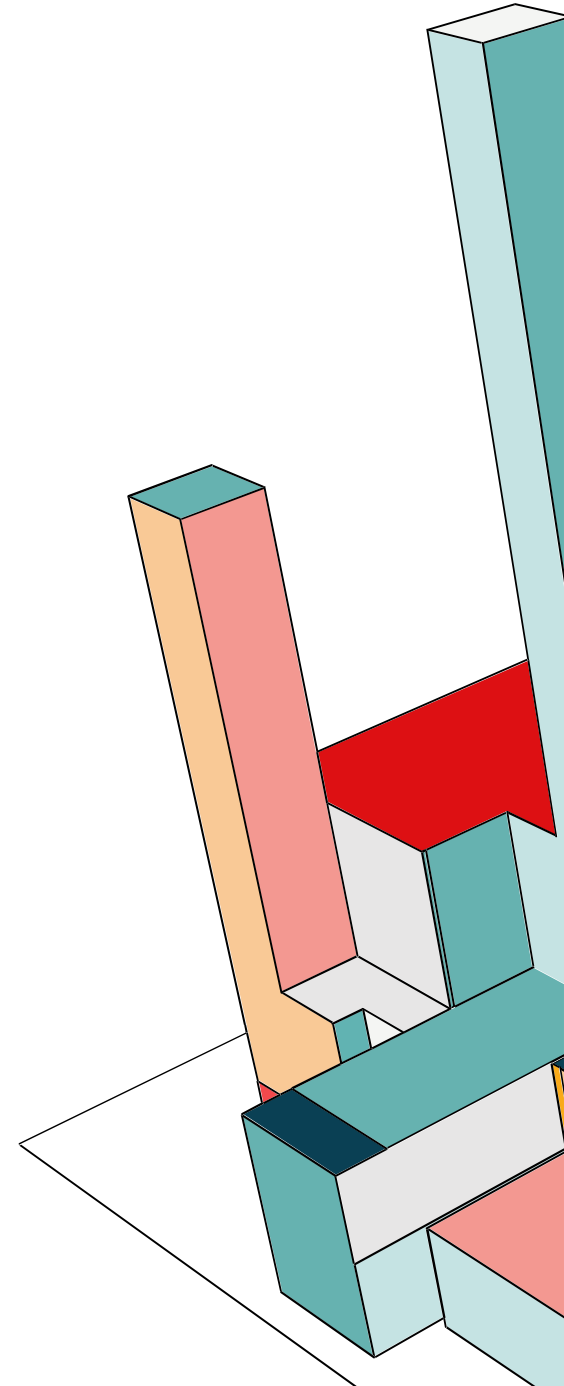
MODEL AGILE

Pendekatan iteratif dan inkremental: aplikasi dibangun sedikit demi sedikit dalam siklus pendek (iterasi atau sprint, umumnya 1–4 minggu). Setiap siklus menghasilkan bagian yang benar-benar dapat dijalankan.

Empat Nilai Agile Manifesto

- Individu dan interaksi lebih dihargai daripada proses dan alat
- Perangkat lunak yang bekerja lebih dihargai daripada dokumentasi yang menyeluruh
- Kolaborasi dengan pelanggan lebih dihargai daripada negosiasi kontrak
- Menanggapi perubahan lebih dihargai daripada mengikuti rencana

Sisi kanan tetap bernilai — hanya saja sisi kiri dinilai lebih berharga.



SCRUM: KERANGKA KERJA AGILE YANG PALING UMUM

Tiga Peran

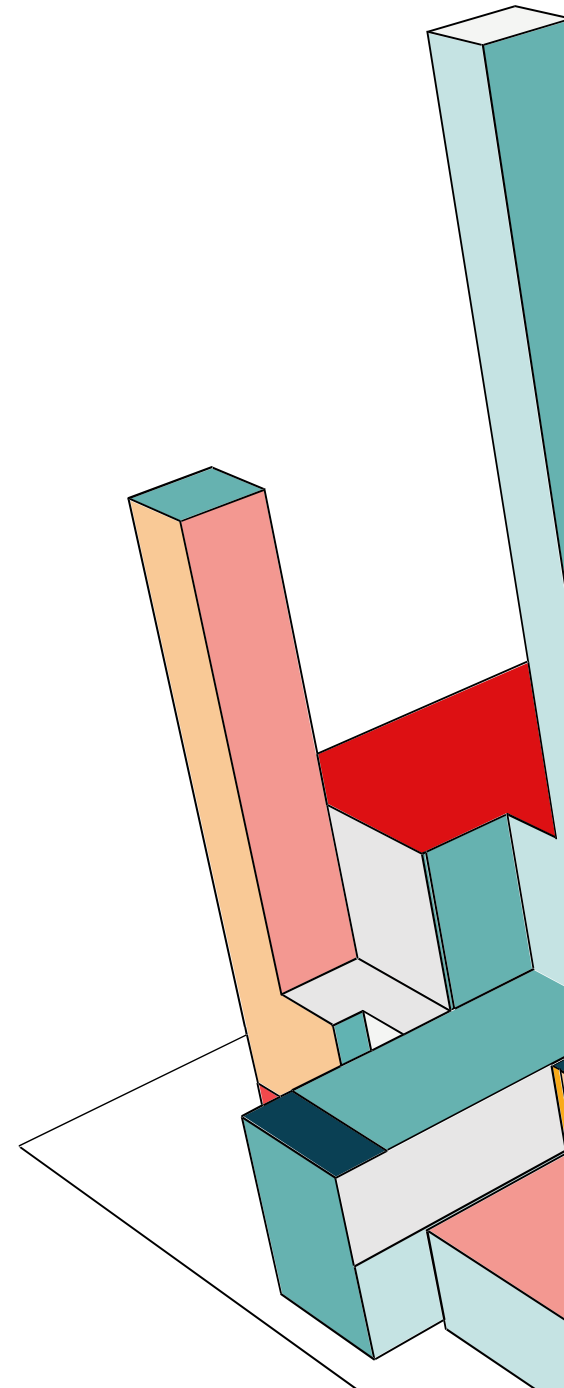
- Product Owner — menentukan prioritas kebutuhan
- Scrum Master — menjaga proses berjalan dan menyingkirkan hambatan
- Development Team — membangun produk

Tiga Artefak

- Product Backlog — daftar seluruh kebutuhan, terurut prioritas
- Sprint Backlog — kebutuhan yang dipilih untuk sprint berjalan
- Increment — hasil kerja yang dapat digunakan di akhir sprint

Empat Acara

- Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective



WATERFALL VS AGILE

Urutan kerja

- Waterfall: linear, sekali jalan | Agile: iteratif, berulang

Kebutuhan

- Waterfall: dibekukan di awal | Agile: boleh berkembang tiap iterasi

Penyerahan hasil

- Waterfall: sekali di akhir proyek | Agile: bertahap setiap iterasi

Umpan balik pengguna

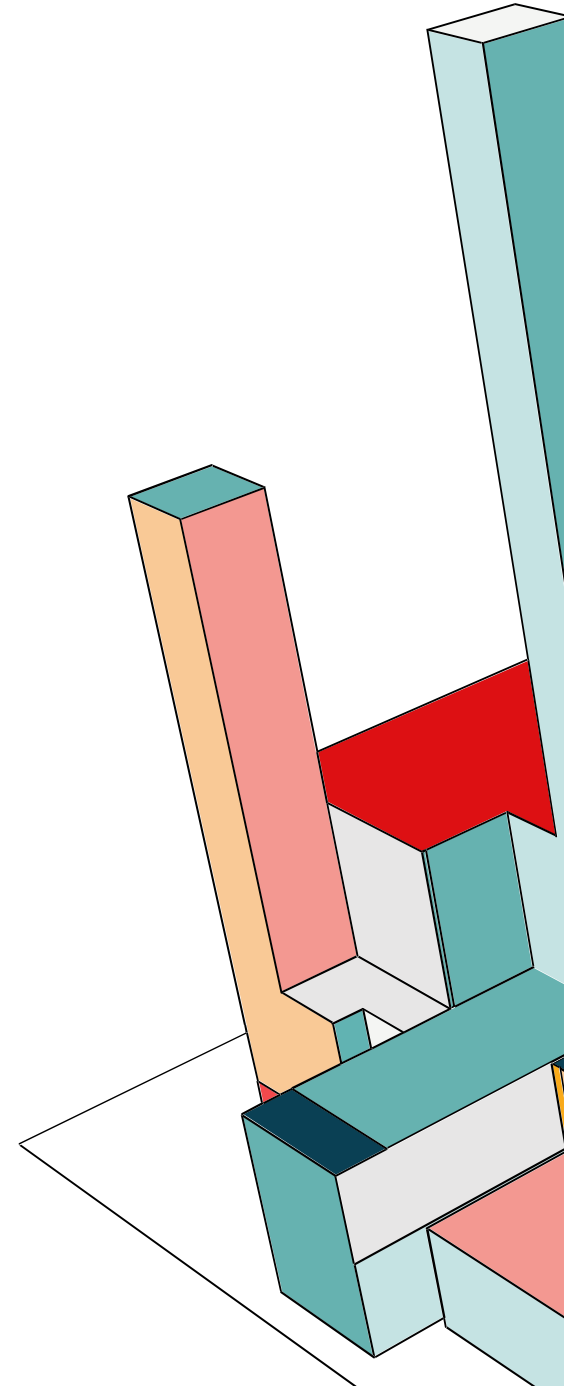
- Waterfall: di akhir | Agile: setiap akhir iterasi

Dokumentasi

- Waterfall: tebal dan formal | Agile: secukupnya, fokus produk berjalan

Biaya perubahan

- Waterfall: tinggi | Agile: relatif rendah



MEMILIH MODEL UNTUK PROYEK ANDA

Tidak ada model yang unggul secara mutlak. Yang menentukan adalah kesesuaian dengan konteks proyek.

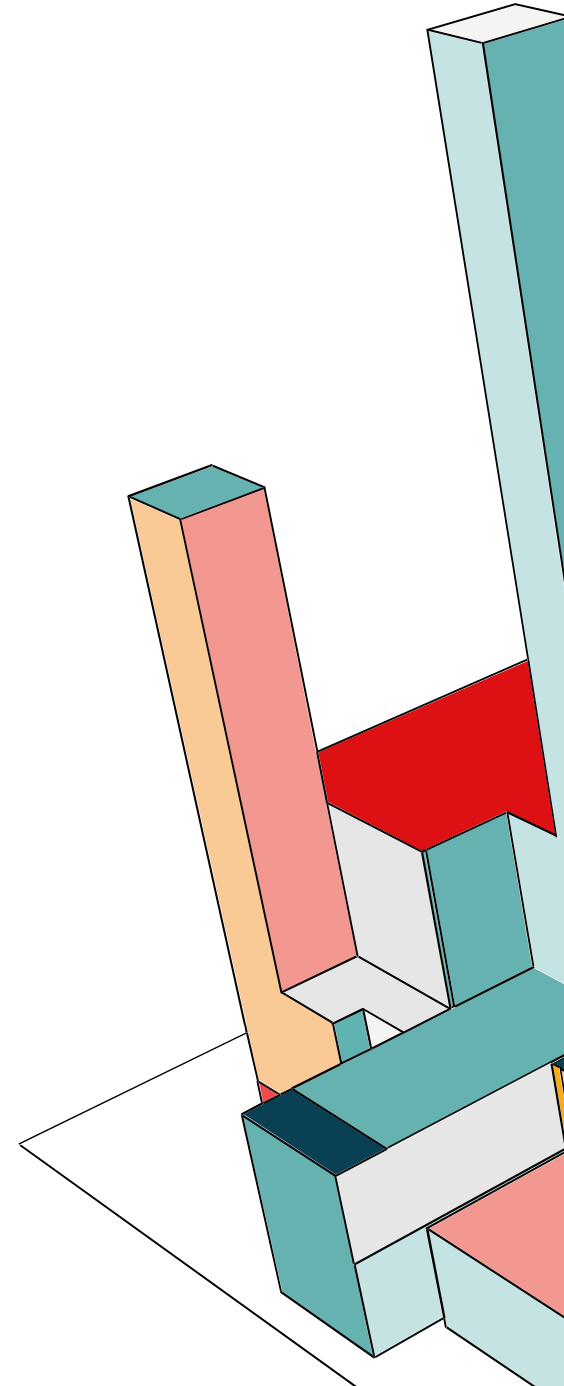
Pilih Waterfall bila:

- Ruang lingkup sudah pasti dan tidak akan berubah
- Terikat kontrak, regulasi, atau standar yang mengharuskan dokumentasi lengkap
- Teknologi dan domain masalah sudah dikuasai tim

Pilih Agile bila:

- Kebutuhan masih akan berkembang selama pengerjaan
- Pengguna tersedia untuk memberi umpan balik secara berkala
- Produk relatif baru dan mengandung ketidakpastian

¹¹
Proyek PjBL pada mata kuliah ini menggunakan pendekatan Agile: fitur dibangun bertahap setiap pertemuan, didemonstrasikan berkala, dan lingkungannya boleh

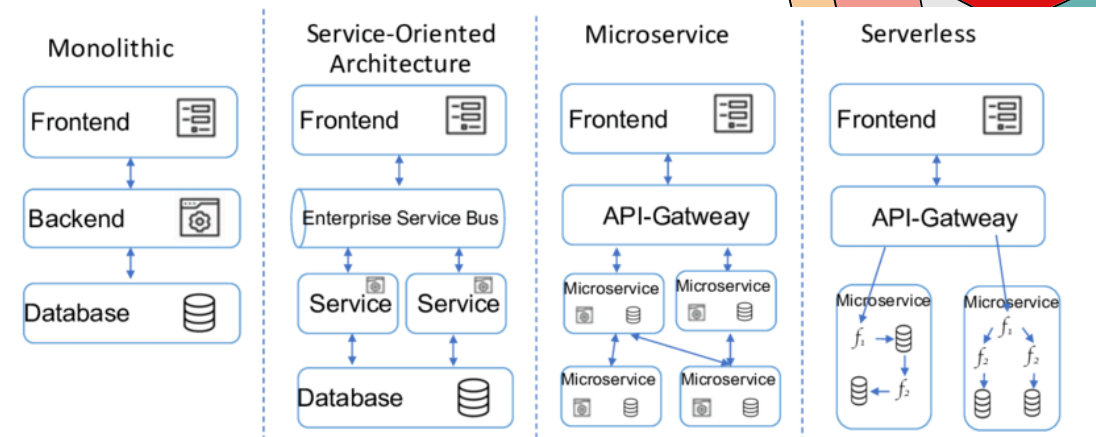
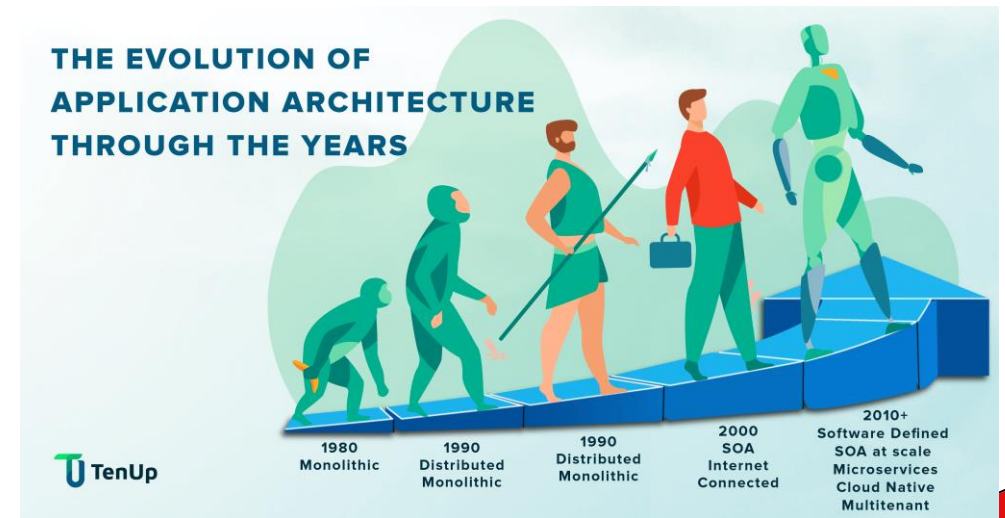


PENGANTAR ARSITEKTUR APLIKASI WEB MODERN



EVOLUSI ARSITEKTUR WEB (EVOLUTION OF WEB ARCHITECTURE):

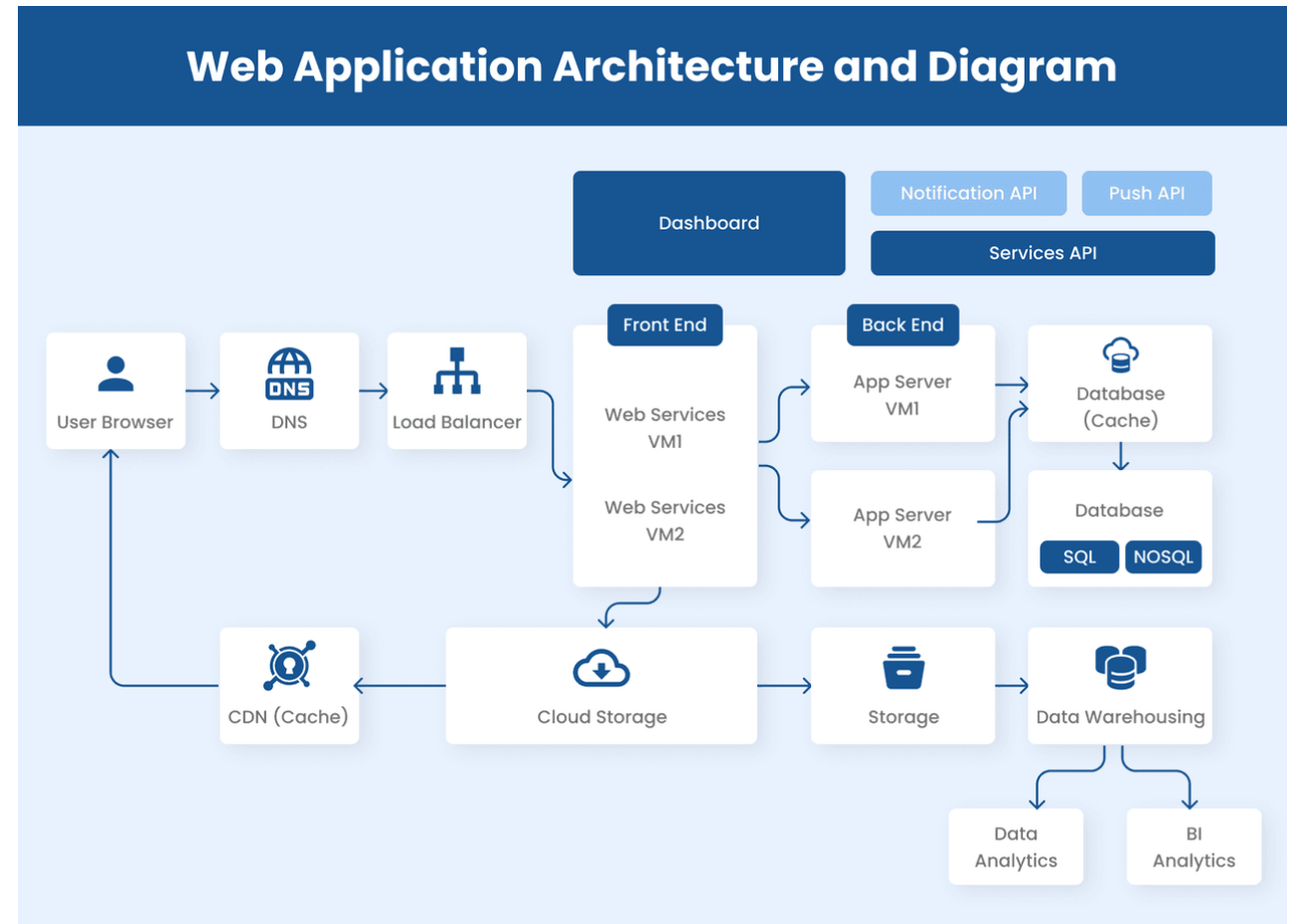
- Awal 1990-an: Situs web statis dengan HTML sederhana
- Akhir 1990-an: Server-side scripting (seperti PHP, ASP) untuk konten dinamis
- Awal 2000-an: Arsitektur tiga tingkat (presentasi, logika bisnis, data)
- Pertengahan 2000-an: AJAX untuk interaksi asinkron
- 2010-an: Single Page Applications (SPA) dan arsitektur berbasis API
- Saat ini: Arsitektur mikro, serverless, dan Progressive Web Apps (PWA)



KOMPONEN UTAMA APLIKASI WEB MODERN

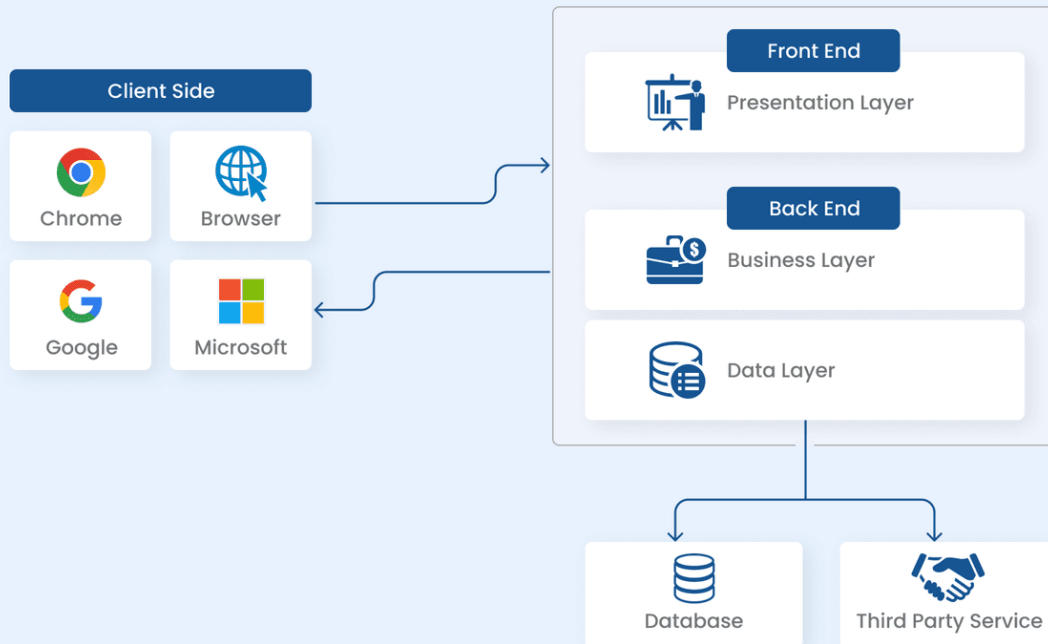
- Front-end: HTML, CSS, JavaScript, framework seperti React atau Vue.js
- Back-end: Server-side languages (misalnya Python, Java, Node.js), database
- API: RESTful atau GraphQL untuk komunikasi client-server
- Caching: Untuk meningkatkan performa
- Content Delivery Networks (CDN): Untuk distribusi konten global
- Authentication and Authorization systems
- Monitoring and Logging tools

(KEY COMPONENTS OF MODERN WEB APPLICATIONS)

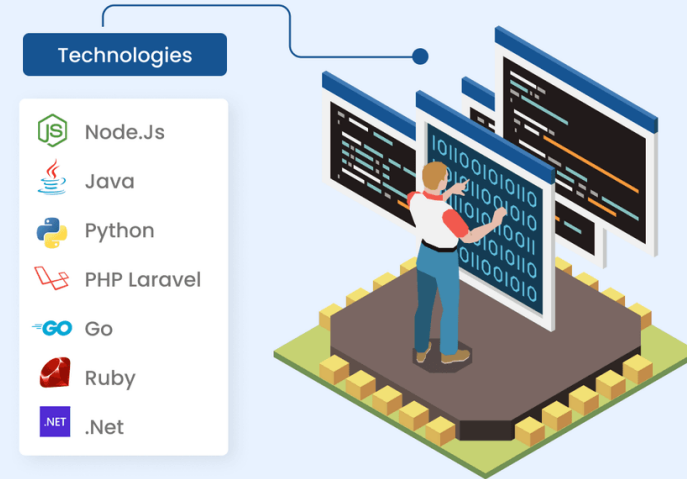


TECHNOLOGIES USE

Standard Web Application Architecture



Back-end Technologies



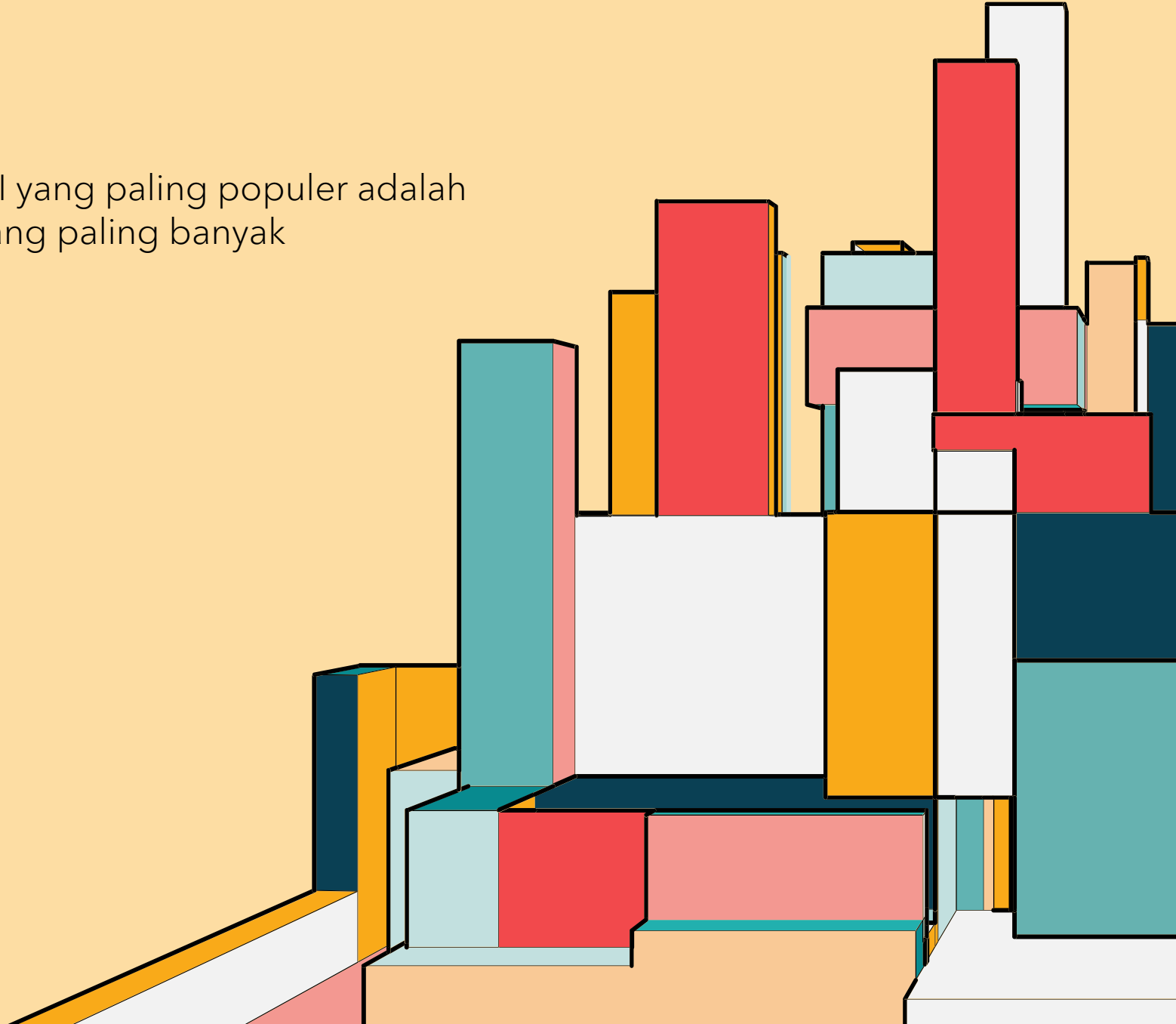
Front-end Technologies



API

Beberapa teknologi pengembangan API yang paling populer adalah .Net dan Java. Berikut ini adalah 4 API yang paling banyak digunakan.

- RESTful API
- SOAP
- XML-RPC
- JSON-RPC



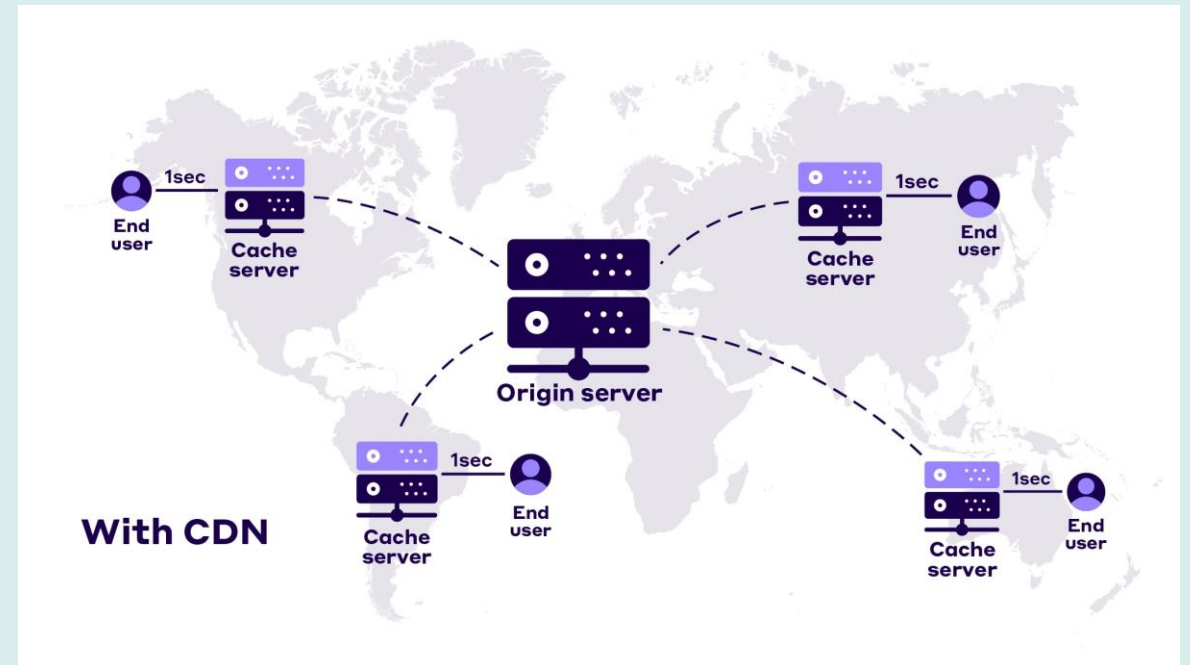
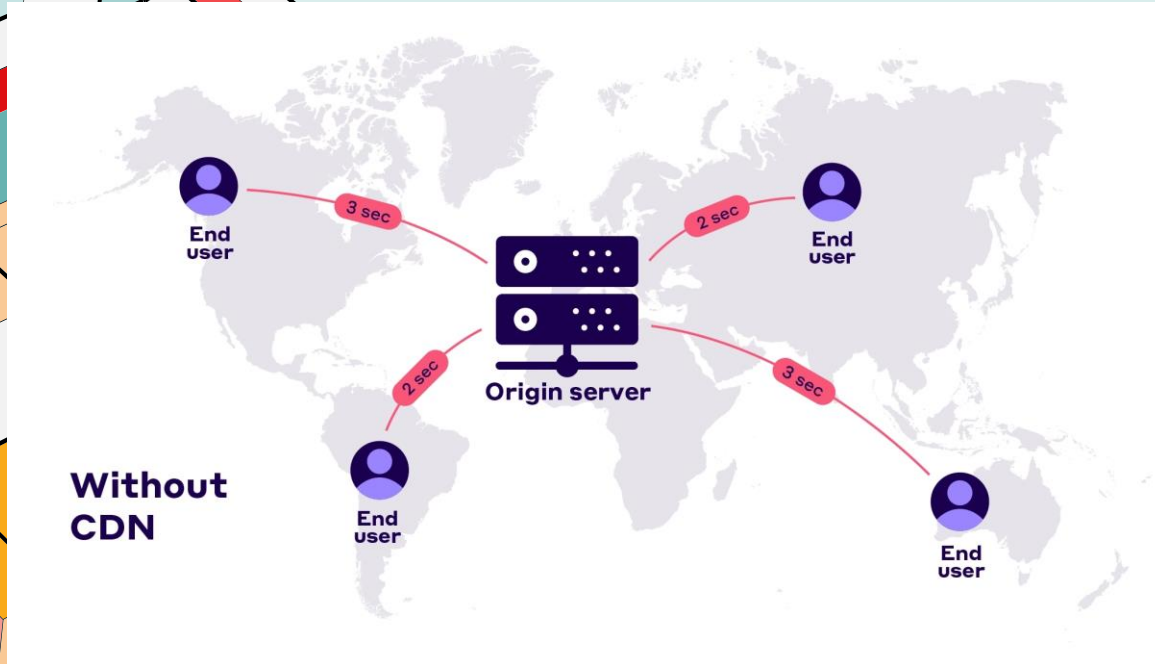


CACHING

There are four types of caching systems:

- Application Server Cache
- Global Cache
- Distributed Cache
- Content Delivery Network

CDN



AUTHENTICATION AND AUTHORIZATION SYSTEMS

Teknik Autentikasi Populer-

1. Otentikasi Berbasis Kata Sandi
2. Autentikasi Tanpa Kata Sandi
3. 2FA/MFA (Autentikasi Dua Faktor/Autentikasi Multifaktor)
4. [Single Sign On \(SSO\)](#)
5. Otentikasi sosial





OTORISASI

Teknik Otorisasi Populer-

- Kontrol Akses Berbasis Peran (RBAC)
- [Otorisasi token web JSON \(JWT\)](#)
- Otorisasi SAML
- Otorisasi OpenID
- Otorisasi OAuth 2.0

MONITORING AND LOGGING TOOLS

This is

TREN TERKINI DALAM ARSITEKTUR WEB

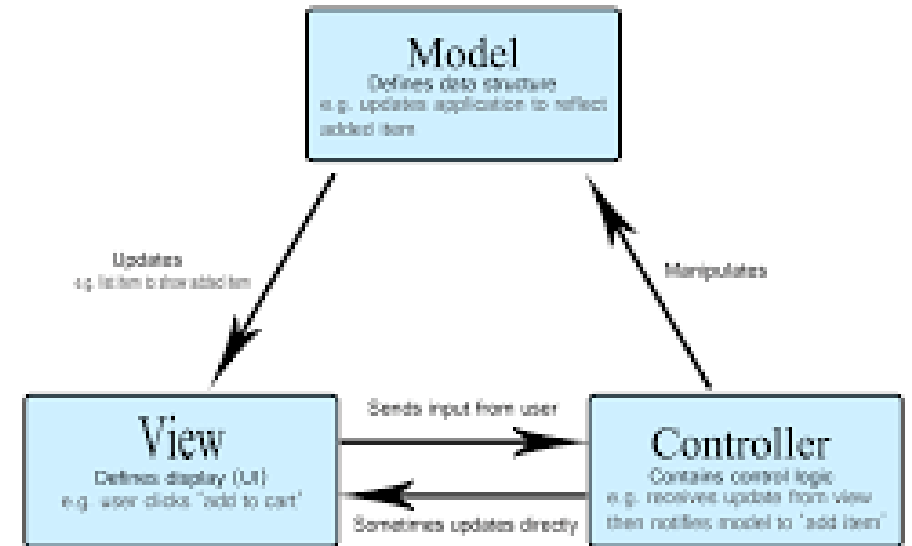
- Microservices: Memecah aplikasi menjadi layanan-layanan kecil yang independen
- Serverless Architecture: Menjalankan kode tanpa mengelola infrastruktur server
- Progressive Web Apps (PWA): Aplikasi web yang berfungsi seperti aplikasi native
- JAMstack: JavaScript, APIs, dan Markup pre-rendered
- Headless CMS: Memisahkan manajemen konten dari presentasi
- Edge Computing: Memproses data lebih dekat ke sumber permintaan
- AI dan Machine Learning Integration: Untuk personalisasi dan otomatisasi
- WebAssembly: Menjalankan kode berkinerja tinggi di browser

MODEL-VIEW- CONTROLLER (MVC)



APA ITU MVC?

- Model-View-Controller atau MVC adalah sebuah metode untuk membuat sebuah aplikasi dengan memisahkan data dari tampilan dan cara bagaimana memprosesnya.
- Pola MVC adalah pola desain perangkat lunak untuk aplikasi web.
- Kerangka kerja MVC memisahkan aplikasi menjadi tiga komponen utama: model, tampilan, dan pengontrol.
- Model digunakan untuk menyimpan data dan logika bisnis; view menampilkan data dari model di layar; dan Controller mengelola interaksi pengguna dengan kedua komponen ini.

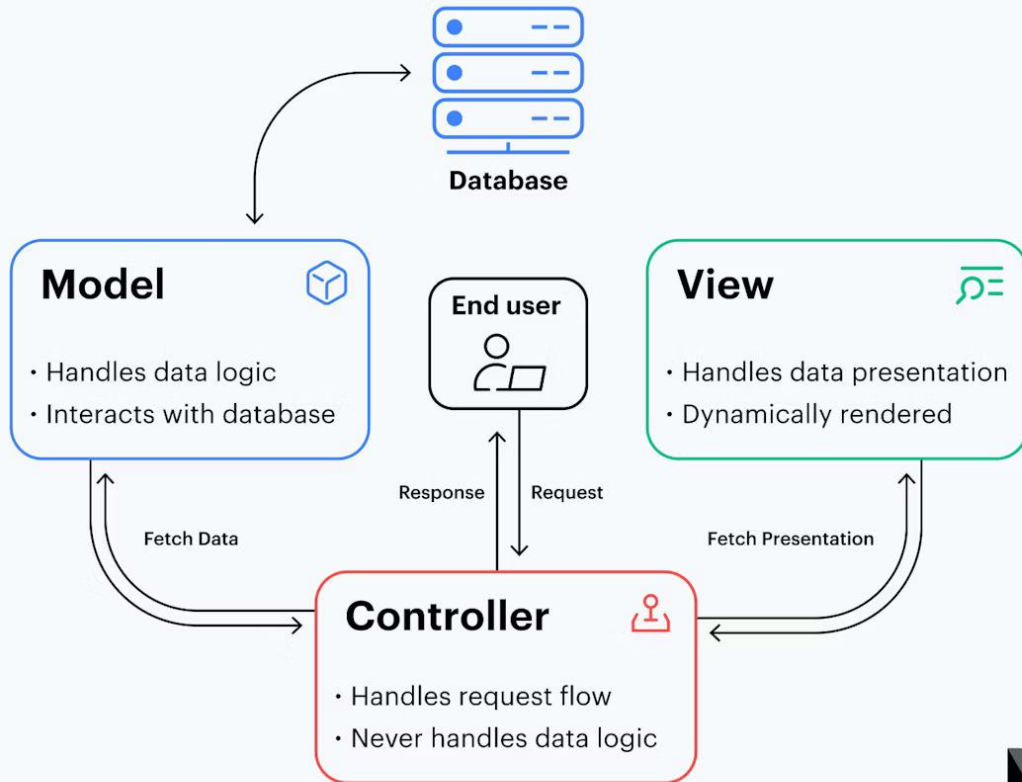


SEJARAH MVC

- Model View Controller pertama sekali dipublikasikan oleh [peneliti XEROX PARC](#) yang bekerja dalam pembuatan [bahasa pemrograman Smalltalk](#) sekitar [tahun](#) 1970-1980.
- Kerangka kerja MVC pertama kali diperkenalkan oleh Trygve Reenskaug pada tahun 1979 sebagai cara untuk mengelola interaksi antara pengguna dan komputer dan masih digunakan sampai sekarang.



MVC Architecture



MVC ARCHITECTURE

- Kerangka kerja MVC adalah arsitektur perangkat lunak yang memisahkan antarmuka pengguna (UI), penyimpanan data, dan logika bisnis menjadi tiga komponen terpisah.
- Antarmuka (UI) : View
- Penyimpanan Data : Model
- Logika Bisnis : Controller

KOMPONEN MODEL

Tanggung jawab: merepresentasikan data dan aturan bisnis aplikasi.

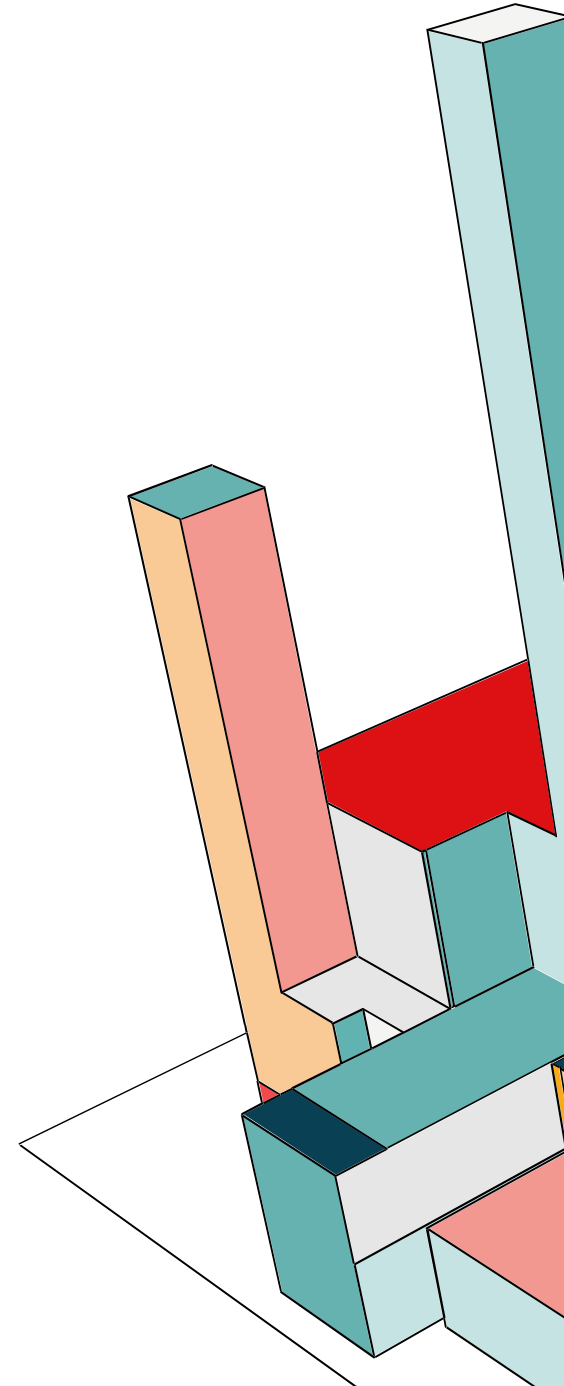
Yang dikerjakan Model

- Mendefinisikan struktur data dan relasi antar entitas
- Berkomunikasi dengan basis data: mengambil, menyimpan, mengubah, menghapus
- Memuat validasi dan perhitungan yang melekat pada data tersebut

Yang bukan urusan Model

- Menghasilkan HTML atau mengatur tampilan
- Mengetahui adanya permintaan HTTP atau sesi pengguna

Pada Laravel: folder `app/Models/`, Eloquent ORM, dan berkas migration di `database/migrations/`



KOMPONEN VIEW

Tanggung jawab: menyajikan data yang sudah disiapkan kepada pengguna.

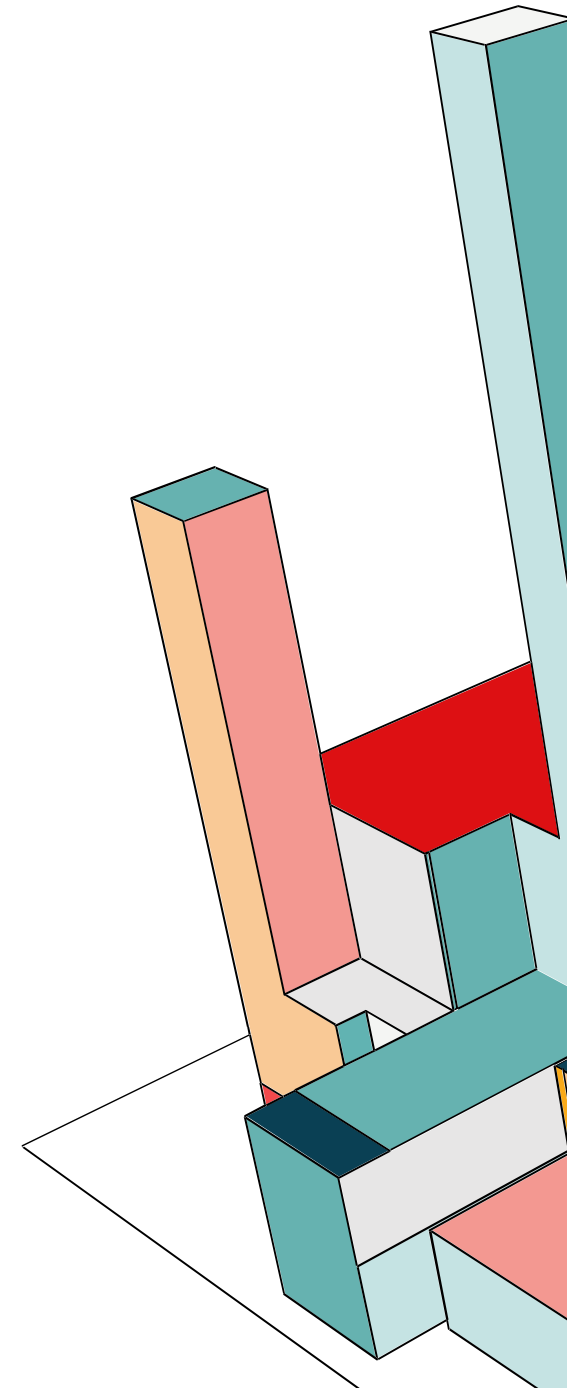
Yang dikerjakan View

- Menampilkan data yang diterima dari Controller
- Menyusun struktur halaman: HTML, tata letak, komponen antarmuka
- Memuat logika presentasi sederhana: perulangan daftar, kondisi tampil atau sembunyi

Yang bukan urusan View

- Melakukan kueri langsung ke basis data
- Memuat aturan bisnis seperti perhitungan denda atau penentuan status

Pada Laravel: folder resources/views/ dengan berkas template Blade (.blade.php)



KOMPONEN CONTROLLER

Tanggung jawab: mengoordinasikan alur antara permintaan pengguna, Model, dan View.

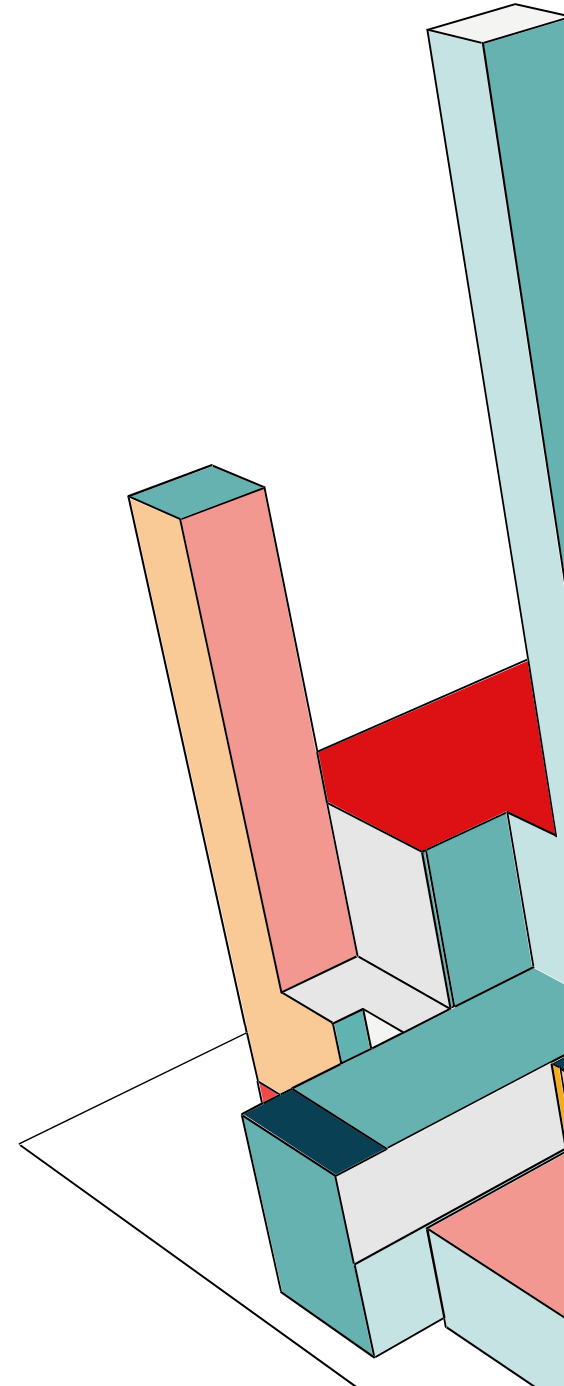
Yang dikerjakan Controller

- Menerima permintaan yang diarahkan oleh sistem routing
- Memvalidasi masukan yang dikirim pengguna
- Memanggil Model untuk mengambil atau mengubah data
- Memilih View yang sesuai dan meneruskan data kepadanya
- Mengembalikan respons: halaman, pengalihan (redirect), atau JSON

Yang bukan urusan Controller

- Menyimpan logika bisnis yang panjang — pindahkan ke Model

Pada Laravel: folder `app/Http/Controllers/`



PEMBAGIAN TANGGUNG JAWAB

Model

- Boleh: kueri basis data, aturan bisnis, validasi data | Tidak boleh: menghasilkan HTML

View

- Boleh: menampilkan data, perulangan, kondisi tampilan | Tidak boleh: kueri basis data

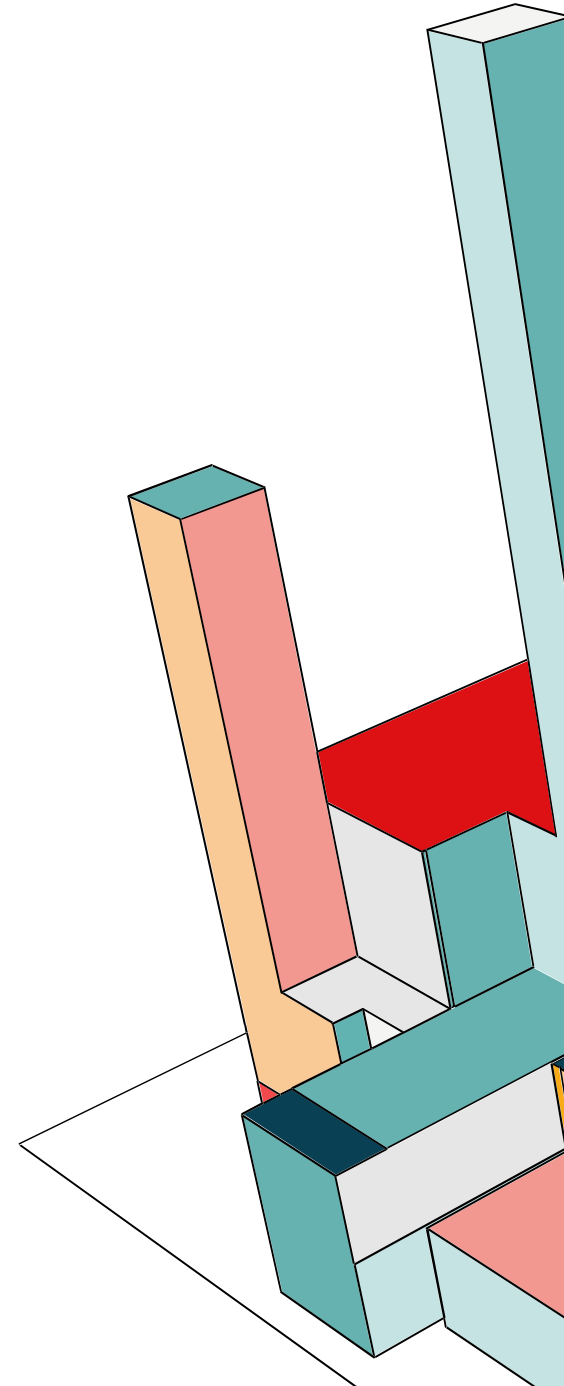
Controller

- Boleh: validasi masukan, memanggil Model, memilih View | Tidak boleh: logika bisnis panjang

Uji pemahaman Anda

Jika seluruh tampilan aplikasi diganti dari HTML biasa menjadi antarmuka baru, kode bagian mana yang seharusnya ikut berubah?

30 Jawaban yang benar: hanya View. Bila Model dan Controller ikut berubah, berarti pemisahan tanggung jawabnya belum benar.



KEUNTUNGAN DAN KEKURANGAN MVC

Clean and organized code

Easy to test and debug

Improved scalability and flexibility

Improved team productivity and collaboration

Increased complexity

Steep learning curve

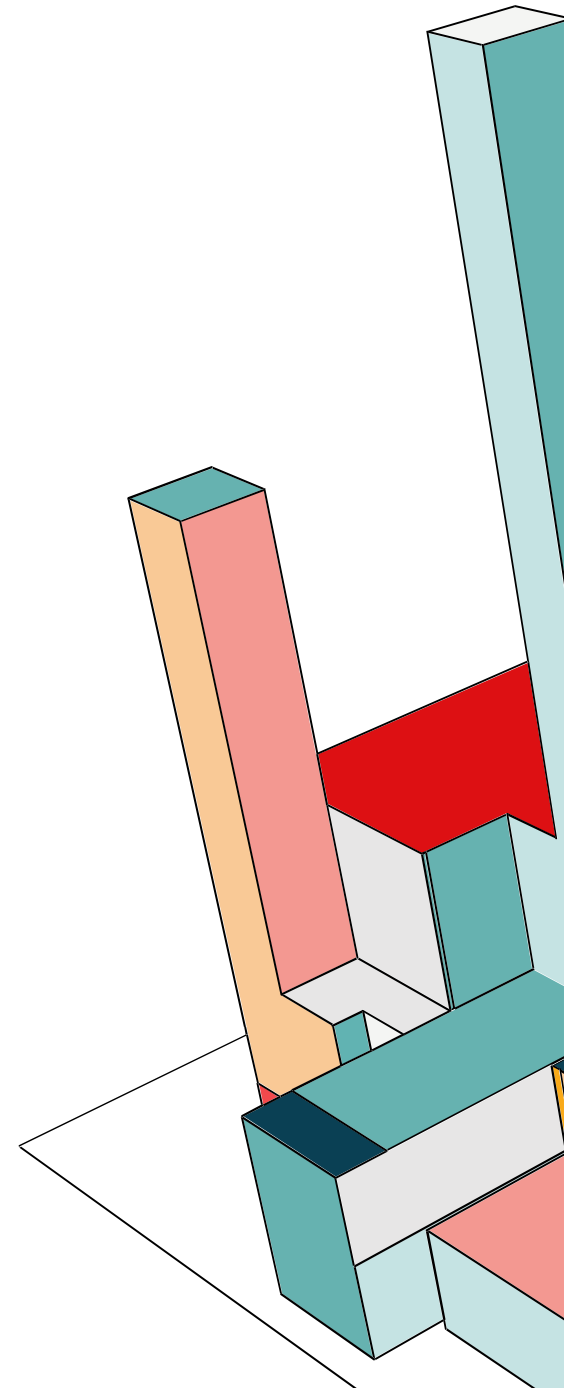
Potential performance issues

Limited control over the user interface

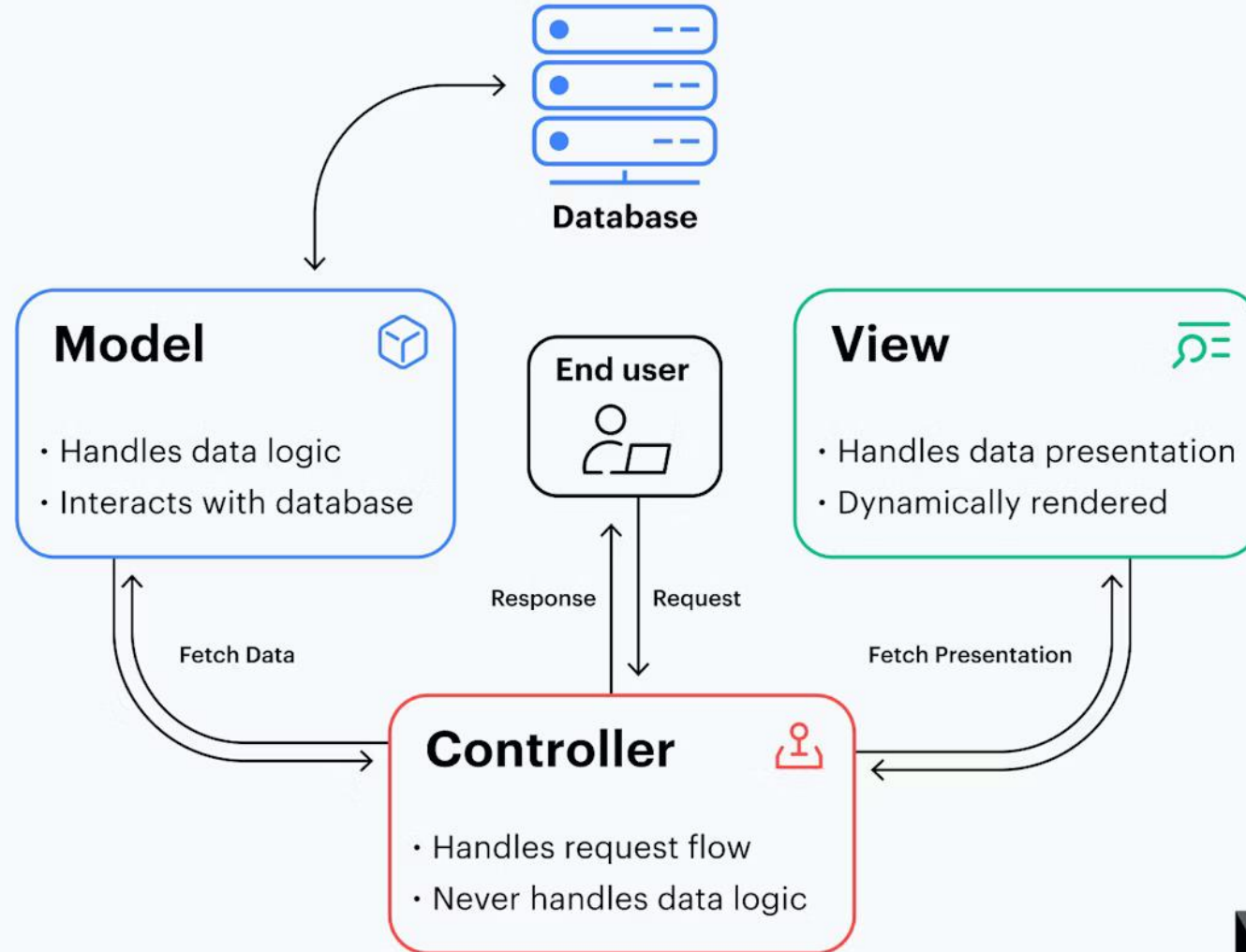
IMPLEMENTASI MVC DALAM APLIKASI WEB MODERN

- **Contoh Framework MVC**

- ASP.NET MVC
- Ruby on Rails
- Laravel (PHP)
- Spring MVC (Java)



MVC Architecture



ALUR PERMINTAAN PADA APLIKASI MVC

1. Permintaan Pengguna

- Pengguna mengklik tautan atau mengirim formulir; peramban mengirim HTTP request ke server

2. Routing ke Controller

- Sistem routing membaca URL dan metode HTTP, lalu mengarahkan ke Controller yang sesuai

3. Interaksi Controller dengan Model

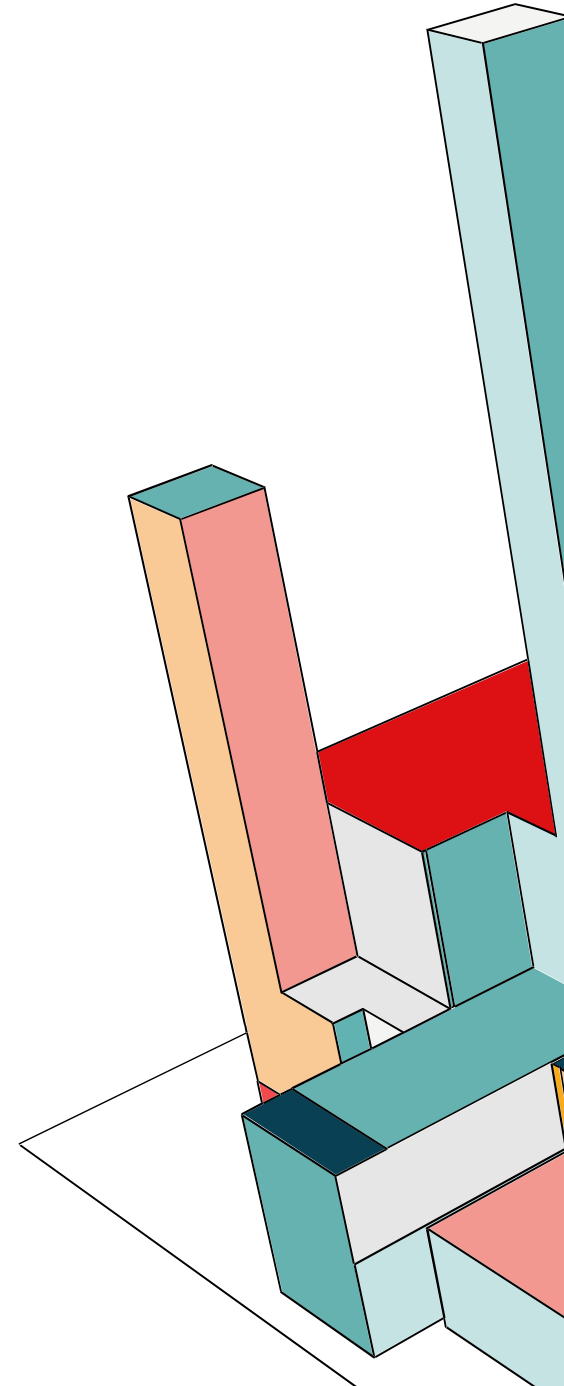
- Controller memproses masukan, meminta data ke Model; Model berkomunikasi dengan basis data dan mengembalikan hasilnya

4. Pembaruan View

- Controller memilih View yang sesuai dan meneruskan data dari Model; View merender tampilan

5. Respons ke Pengguna

- **34** Hasil render dikirim sebagai respons HTTP, dan peramban menampilkan halaman kepada pengguna



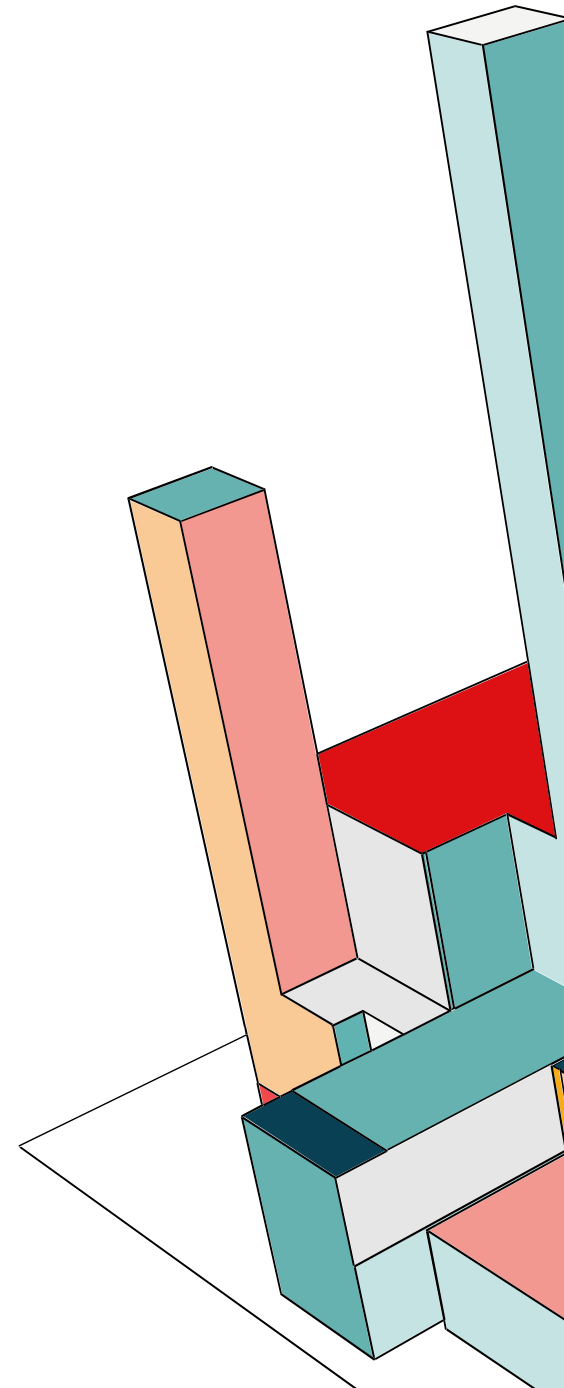
ALUR PENGEMBANGAN FITUR BERBASIS MVC

Urutan kerja yang dianjurkan saat menambahkan satu fitur baru:

1. Tentukan kebutuhan fitur dan data apa saja yang terlibat
2. Buat migration dan Model — siapkan struktur datanya lebih dulu
3. Daftarkan route untuk alamat URL fitur tersebut
4. Tulis Controller — ambil data dari Model, teruskan ke View
5. Buat View untuk menampilkan data kepada pengguna
6. Tambahkan validasi masukan dan penanganan pesan error
7. Uji melalui peramban; bila gagal, periksa pesan error dan berkas log

Kaitan dengan modul praktikum

Route dan Controller dibahas pada Modul 3; View pada Modul 4; Model dan migration pada Modul 5; operasi CRUD pada Modul 6; validasi pada Modul 7.



BEST PRACTICES DALAM MVC

1. Thin Controllers, Fat Models

- Controller:
 - Fokus pada penanganan request dan response
 - Minimal logika bisnis
 - Bertindak sebagai "penghubung" antara Model dan View
- Model:
 - Berisi sebagian besar logika bisnis
 - Menangani validasi data
 - Melakukan operasi kompleks pada data

2. Pemisahan Concerns

- Setiap komponen memiliki tanggung jawab yang jelas dan terbatas
- Model: Logika bisnis dan manajemen data
- View: Presentasi dan formatting data
- Controller: Koordinasi alur aplikasi
- Menghindari duplikasi kode dan meningkatkan reusability

3. Penggunaan View Models

- Objek khusus untuk menyimpan data yang akan ditampilkan di View
- Memisahkan logika presentasi dari Model domain
- Meningkatkan fleksibilitas dan maintainability

1. THIN **CONTROLLERS, FAT** **MODELS**

2. PEMISAHAN **CONCERNS**

3. PENGGUNAAN **VIEW MODELS**

THANK YOU

